Testing of a Cryptographic Hash Function: SHA-256 Analysis

Name: Irmak Yılmaz
Student Number: E-379

1. Selection of the Hash Function

Selected Hash Function: SHA-256 (Secure Hash Algorithm 256-bit)

Reason for Selection: I chose SHA-256 from the Python hashlib library because it is one of the most widely used and secure modern cryptographic hash functions. It generates a fixed 256-bit (32-byte) hash value regardless of the input size. It is deeply integrated into many secure systems, from digital signatures to blockchain technologies.

2. Preparation of the Testing Environment

Tools Used: I developed a custom testing script in Python. Python's built-in hashlib library was used for hash computations. To perform the bit-level analysis and measure the avalanche effect, I wrote specific functions to convert the hash outputs into binary strings, calculate the Hamming distance, and measure the Shannon entropy of the bits.

Input Data: The testing environment generates dynamic datasets:

- For the determinism test, a static string is hashed multiple times.
- For the avalanche effect, two nearly identical strings (differing by only a single bit) are compared.
- For the performance test, exact file sizes of 1 KB, 100 KB, and 1 MB are generated using random bytes.

3. Conducting the Tests

Using the custom Python script, I calculated the SHA-256 hash values for various input data and evaluated determinism, the avalanche effect, performance, and bit-level entropy.

Here are the precise results obtained from the testing environment:

```
PS C:\Users\23022\OneDrive\Desktop\ders> python hash_test.py
=== Cryptographic Hash Function Tests (SHA-256) ===

--- 1. Determinism Test ---
Message   : ErasmusCryptographyProject
Hash 1    : be76f6040d207f183fc7be189c05147aa1d413d6dcd48688a9ec966c61862d30
Hash 2    : be76f6040d207f183fc7be189c05147aa1d413d6dcd48688a9ec966c61862d30
Result    : SUCCESS (Both hashes are strictly identical)

--- 2. Avalanche Effect Test ---
Input 1   : 'Data'
Input 2   : 'data' (1 bit changed)
Bits flipped in Hash : 126 out of 256
Avalanche Effect     : 49.22%

--- 3. Performance Test ---
Hashing 1 KB file took: 0.000016 seconds
Hashing 100 KB file took: 0.000132 seconds
Hashing 1 MB file took: 0.001231 seconds

--- 4. Bit-Level Analysis (Entropy & Distribution) ---
Total Bits Evaluated : 256
Number of 0s         : 121
Number of 1s         : 135
Ideal Entropy        : 1.0 (Perfect randomness)
Calculated Entropy   : 0.997842
```

4. Analysis of Results

Determinism: The test successfully demonstrated that hashing the identical string multiple times consistently produces the exact same 256-bit hexadecimal output. This strict determinism is the foundational property of cryptographic hashes, allowing systems to reliably verify passwords, digital signatures, and data integrity over time.

Avalanche Effect: The avalanche effect test yielded an outstanding result of 49.22%. Changing merely a single bit in the input string (from uppercase 'D' to lowercase 'd') caused 126 out of the 256 bits in the resulting hash to flip. This behavior is remarkably close to the theoretical ideal of 50%. It ensures that even the slightest alteration in a file completely randomizes the output, making it mathematically infeasible for an attacker to predict the input based on the hash.

Performance and Efficiency: The execution time scales linearly with the data size, but remains incredibly fast overall. Hashing a 1 MB dataset took approximately 0.0012 seconds. In modern software engineering and data science architectures, this efficiency proves that SHA-256 can be seamlessly integrated into data pipelines to verify the integrity of large datasets during transmission (e.g., from databases to AI models) without causing any computational bottlenecks.

Statistical Characteristics (Entropy): The bit-level analysis reveals a highly balanced distribution of zeros (121) and ones (135). Furthermore, the calculated Shannon entropy is 0.9978, which is exceptionally close to the perfect randomness score of 1.0. This high entropy proves that the SHA-256 output lacks predictable patterns and resembles true random noise, offering robust security against statistical cryptanalysis.